Date:30/10/2026

12. Developing Multiple Linear Regression from Scratch

Procedure:

Step 1: Import essential Python libraries including NumPy for matrix operations, Pandas for data handling, and Matplotlib for plotting visual graphs.
Step 2: Load the dataset and split it into feature input matrix X and target output vector y, then count total training examples m.
Step 3: Create a scatter plot of house size versus price to visually inspect the data distribution and check for linear relationships.
Step 4: Apply feature scaling and mean normalization to bring input values into similar ranges, then prepend a column of ones to X.
Step 5: Define the cost calculation function using matrix dot products to measure the mean squared error between predictions and target labels.
Step 6: Build the gradient descent function to iteratively update theta parameters based on errors and the learning rate alpha.
Step 7: Initialize weights to zero, execute gradient descent for 400 iterations, and verify that the calculated cost decreases continuously.
Step 8: Run gradient descent across multiple learning rates and plot all convergence curves on one graph to find the optimal alpha value.
Step 9: Combine target values and predictions into a matrix to review output accuracy and calculate the custom R-squared score.
Step 10: Train Scikit-Learn's LinearRegression model on the same data to validate that custom coefficients and R-squared score match standard library results.

Solution:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import linear_model
df = pd.read_csv(r"C:\Users\Lenovo\Downloads\house_size_price_dataset.csv")
X = df.values[:, 0:2]
y = df.values[:, 2]
m = len(y)

print('Total no of training examples (m) = %s \n' % (m))
for i in range(5):
    print('x = ', X[i], 'y =', y[i])
X1 = df["size(in square feet)"]
Y1 = df[" price"]

plt.figure(figsize=(10, 6))
plt.scatter(X1, Y1, color='red', marker='+')
plt.grid(True)
plt.xlabel('Size (in square feet)')
plt.ylabel('Price')
plt.title('Scatter plot of training data (Size vs Price)')
plt.show()
def feature_normalize(X):
    mu = np.mean(X, axis=0)
    sigma = np.std(X, axis=0, ddof=1)
    X_norm = (X - mu) / sigma
    return X_norm, mu, sigma

X_norm, mu, sigma = feature_normalize(X)

print('mu = ', mu)
print('sigma = ', sigma)
print('X_norm (first 5) = \n', X_norm[:5])
X_normalized_with_intercept = np.hstack((np.ones((m, 1)), X_norm))
def compute_cost(X, y, theta):
    predictions = X.dot(theta)
    errors = np.subtract(predictions, y)
    J = 1 / (2 * m) * errors.T.dot(errors)
    return J
```

```
def gradient_descent(X, y, theta, alpha, iterations):
    cost_history = np.zeros(iterations)
    for i in range(iterations):
        predictions = X.dot(theta)
        errors = np.subtract(predictions, y)
        sum_delta = (alpha / m) * X.transpose().dot(errors)
        theta = theta - sum_delta
        cost_history[i] = compute_cost(X, y, theta)
    return theta, cost_history
iterations = 400

theta_init = np.zeros(3)
theta_1, cost_history_1 = gradient_descent(X_normalized_with_intercept, y, theta_init.copy(), 0.005, iterations)
theta_2, cost_history_2 = gradient_descent(X_normalized_with_intercept, y, theta_init.copy(), 0.01, iterations)
theta_3, cost_history_3 = gradient_descent(X_normalized_with_intercept, y, theta_init.copy(), 0.02, iterations)
theta_4, cost_history_4 = gradient_descent(X_normalized_with_intercept, y, theta_init.copy(), 0.03, iterations)
theta_5, cost_history_5 = gradient_descent(X_normalized_with_intercept, y, theta_init.copy(), 0.15, iterations)
theta_6, cost_history_6 = gradient_descent(X_normalized_with_intercept, y, theta_init.copy(), 1.32, iterations)
plt.figure(figsize=(10, 6))
plt.plot(range(1, iterations + 1), cost_history_1, color='purple', label='alpha = 0.005')
plt.plot(range(1, iterations + 1), cost_history_2, color='red', label='alpha = 0.01')
plt.plot(range(1, iterations + 1), cost_history_3, color='green', label='alpha = 0.02')
plt.plot(range(1, iterations + 1), cost_history_4, color='yellow', label='alpha = 0.03')
plt.plot(range(1, iterations + 1), cost_history_5, color='blue', label='alpha = 0.15')
plt.plot(range(1, iterations + 1), cost_history_6, color='brown', label='alpha = 1.32')
plt.grid(True)
plt.xlabel("Number of iterations")
plt.ylabel("Cost (J)")
plt.title("Effect of Learning Rate On Convergence of Gradient Descent")
plt.legend()
plt.show()
print('Final optimized weights (theta_5) =', theta_5)
predict = np.round(X_normalized_with_intercept.dot(theta_5), 1)
print("\nFirst 5 Actual vs Custom Predicted prices:")
Act_Predict = np.stack((y, predict), axis=1)
print(Act_Predict[:5,])
def r2score(pred, actual):
    rss = np.sum((pred - actual) ** 2)
    tss = np.sum((actual - actual.mean()) ** 2)
    return 1 - (rss / tss)

custom_r2 = np.round(r2score(predict, y), 4)
print(f'\nCustom Model R2 Score: {custom_r2}')
model_ols = linear_model.LinearRegression()
model_ols.fit(X_normalized_with_intercept, y)
print('\nSklearn coef= ', model_ols.coef_)
print('Sklearn intercept= ', model_ols.intercept_)
y_pred_sklearn = model_ols.predict(X_normalized_with_intercept)
print('Sklearn Model Score (R2): ', np.round(model_ols.score(X_normalized_with_intercept, y), 4))
```

Total no of training examples (m) = 30

x = [650 1] y = 145000

x = [800 2] y = 182000

x = [900 2] y = 194000

x = [1000 2] y = 205000

x = [1100 2] y = 221000

mu = [1956.66666667 3.46666667]

sigma = [777.12260309 1.10588811]

X_norm (first 5) =

[[-1.68141637 -2.23048485]

[-1.48839663 -1.32623423]

[-1.35971681 -1.32623423]

[-1.23103699 -1.32623423]

[-1.10235716 -1.32623423]]

Final optimized weights (theta_5) = [380333.33333333 122459.99897219 18603.34946026]

First 5 Actual vs Custom Predicted prices:

[[145000. 132932.6]

 [182000. 173391.9]

 [194000. 189150.]

 [205000. 204908.1]

 [221000. 220666.3]]

Custom Model R2 Score: 0.9986

Sklearn coef= [0.   124974.924259   16088.42417345]

Sklearn intercept= 380333.3333333333

Sklearn Model Score (R2): 0.9986



